SafeRoute Guardian — Build Checklist (Start to Finish)

How to use: work top to bottom. Every step has **what to build**, **how**, and a **Done when** test. Tags: `[FE]` frontend, `[BE]` backend, `[DATA]` zones/data, `[AI]` Gemini, `[QA]` testing, `[OPS]` setup/deploy.

Suggested parallel workstreams for 4 people (assign as you like):

- **A – Frontend shell:** Phases 3, 4, 9, 14
- **B – Geo + engine:** Phases 5, 6, 7, 8 (the core, most important)
- **C – Backend + AI + SOS:** Phases 10, 11, 12
- **D – Data + QA + demo:** Phases 2, 13, 15, 17

Everything marked *(proposed)* is a design choice not in the pitch deck. Estimated hours are rough, for one person.

Phase 0 — Lock decisions and accounts (1–2 h) `[OPS]`

- [ ] **0.1 Lock the demo story.** One origin, one destination, one time (e.g., 10:30 PM) in Bengaluru where the fastest route crosses a Red zone and a detour exists. Everything else supports this story.
- [ ] **0.2 Create accounts and keys.**
  - Mapbox account → copy the **default public token** (starts `pk.`). Later restrict it to your deployed URLs.
  - Google AI Studio → create a **Gemini API key**.
  - GitHub repo (private until the hackathon ends).
  - Vercel (client) and Render or Railway (server) accounts.
- [ ] **0.3 Agree on the tech stack:** React + Tailwind CSS, Mapbox GL JS + Directions API, Turf.js, Gemini API, Node.js + Express. I recommend **Vite** to scaffold React (fastest setup); the PRD table lists React/Tailwind only, so swap if you prefer.
- [ ] **0.4 Agree on git rules:** `main` always demo-ready; feature branches `feat/<name>`; small PRs; one person merges.
- [ ] **0.5 Create a shared doc/board** with this checklist as tasks.

Done when: every member can run `git clone`, has the keys in a private channel, and the demo story is

written in one sentence.

Phase 1 — Project scaffold (1–2 h)

`[OPS]`

- [ ] **1.1 Create the monorepo.**

  ```
  mkdir saferoute-guardian && cd
  saferoute-guardian
  git init
  npm create vite@latest client -- --
  template react
  mkdir server shared docs
  ```

- [ ] **1.2 Install client dependencies.**

  ```
  cd client
  npm install mapbox-gl @turf/turf
  zustand
  npm install -D tailwindcss
  @tailwindcss/vite vitest
  ```

  Tailwind v4 setup: add the plugin in `vite.config.js` and put `@import "tailwindcss";` at the top of `src/index.css`. (If you use Tailwind v3, follow its `init` flow instead.)

- [ ] **1.3 Install server dependencies.**

```
cd ../server
npm init -y
npm install express cors dotenv
express-rate-limit @google/genai
```

Set `"type": "module"` in `server/package.json`.

- [ ] **1.4 Environment files** (never commit real keys).
  - `client/.env.local`

    ```
    VITE_MAPBOX_TOKEN=pk.xxxxx
    VITE_API_BASE=http://localhost:3001
    ```

  - `server/.env`

    ```
    PORT=3001
    GEMINI_API_KEY=xxxxx
    GEMINI_MODEL=<a current Gemini Flash
    model name from the Gemini docs>
    CORS_ORIGIN=http://localhost:5173
    ```

  - Add both to `.gitignore`; commit `.env.example` files with blank values.
- [ ] **1.5 Run both servers.** Express serves `GET /api/health → { ok: true }`; the Vite app loads.
- [ ] **1.6 Root scripts** (optional): use `npm-run-all` or two terminals.

Done when: `localhost:5173` shows the Vite page and `localhost:3001/api/health` returns `{"ok":true}`.

Phase 2 — Synthetic risk-zone data (3–4 h) `[DATA]`

Goal: believable, **clearly synthetic** Bengaluru zones that make the demo route cross a Red zone.

- [ ] **2.1 Pick the demo pair.** Open Mapbox/Google Maps and choose an origin and destination about 5–8 km apart (so the fastest route is roughly 18 min by car). Note both coordinates as `[lng, lat]`.
- [ ] **2.2 Look at the fastest route** (Mapbox Directions or Google Maps) and note the path it takes.
- [ ] **2.3 Draw the zones** at https://geojson.io:
  - 1 **Red** polygon straddling the fastest route's middle section.
  - 2–3 **Yellow** polygons near the route or on alternative roads.
  - 3–5 **Green** polygons elsewhere (lower risk), including along the detour road.
  - Add 6–10 filler zones across the city so the overlay looks real.

    - Keep the detour road **clear of Red** so a safe corridor exists.

- [ ] **2.4 Add properties to each feature.**

```
{
  "type": "Feature",
  "properties": {
    "id": "zone_001",
    "name": "Demo Zone A (synthetic)",
    "risk": "red",
    "note": "Poorly lit, isolated
stretch (synthetic)"
  },
  "geometry": { "type": "Polygon",
"coordinates": [[[77.59,12.97],
[77.60,12.97],[77.60,12.98],
[77.59,12.98],[77.59,12.97]]] }
}
```

  `risk` must be exactly `red`, `yellow` or `green`. The `note` field powers the copilot's explanations.

- [ ] **2.5 Save** as `server/data/zones.bengaluru.geojson` and copy to `client/public/zones.bengaluru.geojson` (offline fallback).
- [ ] **2.6 Validate:** polygons are closed (first point = last point), coordinates are `[lng, lat]` (not lat/lng), file parses as JSON, every feature has `id` and `risk`.

- [ ] **2.7 Record the demo constants** in `shared/demo.js`:

```
export const DEMO = {
  origin:      { label: 'Origin name',
coords: [77.0, 12.0] },   // replace
  destination: { label: 'Destination
name', coords: [77.0, 12.0] },   //
replace
  time: '22:30'
};
```

Done when: pasting the file into geojson.io shows the zones in the right place, with a Red zone on top of the fastest route.

Phase 3 — Frontend foundation (4–5 h) `[FE]`

- [ ] **3.1 Layout shell** (`App.jsx`). Desktop: left sidebar (≈380 px) with inputs and results, right side full-height map. Mobile: map on top, bottom sheet for controls. Use Tailwind `flex`, `h-screen`, `md:` breakpoints.
- [ ] **3.2 Visual theme.** Dark UI matching the pitch deck (black background, neon-green accent). Define CSS variables or Tailwind theme colours: `-`

`-accent`, `--risk-red`, `--risk-yellow`, `--risk-green`, `--safe-route`, `--fast-route`.

- [ ] **3.3 Global store** with Zustand (`src/state/useStore.js`):

```
import { create } from 'zustand';
export const useStore = create((set) =>
({
  origin: null, destination: null,
travelTime: new Date(),
  mode: 'driving',
  routes: [],            // [{id,
geometry, durationMin, distanceKm,
analysis, score, cost, label}]
  activeRouteId: null,
  zones: null,
  config: null,
  status: 'idle',        // idle |
loading | ready | error
  error: null,
  copilot: [],           // [{role,
text}]
  setOrigin: (origin) => set({ origin
}),
  setDestination: (destination) =>
set({ destination }),
  setTravelTime: (travelTime) => set({
travelTime }),
  setMode: (mode) => set({ mode }),
  setResults: (routes, activeRouteId)
=> set({ routes, activeRouteId, status:
'ready', error: null }),
  setActive: (activeRouteId) => set({
```

```
activeRouteId }),
  setStatus: (status, error = null) =>
set({ status, error }),
  setZones: (zones) => set({ zones }),
  setConfig: (config) => set({ config
}),
  addCopilot: (m) => set((s) => ({
copilot: [...s.copilot, m] })),
}));
```

- [ ] **3.4 Folder structure** under `client/src/`: `components/`, `lib/` (routing, spatial, riskEngine, api), `state/`, `hooks/`.
- [ ] **3.5 Persistent disclaimer badge** in the corner: "Demo data is synthetic. Scores are not a guarantee of safety."

Done when: the dark shell renders, the store works (log it), and resizing to a phone width keeps the layout usable.

Phase 4 — Map canvas (3–4 h) `[FE]`

- [ ] **4.1** `MapCanvas.jsx` using `mapbox-gl` directly:

```
import { useEffect, useRef } from
'react';
import mapboxgl from 'mapbox-gl';
import 'mapbox-gl/dist/mapbox-gl.css';
```

```
mapboxgl.accessToken =
import.meta.env.VITE_MAPBOX_TOKEN;

export default function MapCanvas({
onReady }) {
  const el = useRef(null);
  useEffect(() => {
    const map = new mapboxgl.Map({
      container: el.current,
      style:
'mapbox://styles/mapbox/dark-v11',
      center: [77.5946, 12.9716],   //
Bengaluru
      zoom: 11,
    });
    map.addControl(new
mapboxgl.NavigationControl(), 'top-
right');
    map.on('load', () => onReady?.
(map));
    return () => map.remove();
  }, []);
  return <div ref={el} className="h-
full w-full" />;
}
```

- [ ] **4.2 Expose the map instance** via a ref/context so other components add layers.
- [ ] **4.3 Zone overlay layer** (after map `load` and zones fetched):

```
map.addSource('zones', { type:
'geojson', data: zones });
map.addLayer({
  id: 'zones-fill', type: 'fill',
source: 'zones',
  paint: {
    'fill-color': ['match', ['get',
'risk'],
      'red', '#ff3b3b', 'yellow',
'#ffd23f', 'green', '#2ecc71', '#888'],
    'fill-opacity': 0.25,
  },
});
map.addLayer({
  id: 'zones-outline', type: 'line',
source: 'zones',
  paint: { 'line-color': ['match',
['get', 'risk'],
    'red', '#ff3b3b', 'yellow',
'#ffd23f', 'green', '#2ecc71', '#888'],
'line-width': 1 },
});
```

- [ ] **4.4 Zone toggle** button (show/hide with `map.setLayoutProperty(id, 'visibility', 'none' | 'visible')`).
- [ ] **4.5 Legend** component: Red = High, Yellow = Moderate, Green = Lower.
- [ ] **4.6 Click on a zone** → popup with `name`, `risk`, `note`.

Done when: the dark map loads on Bengaluru with coloured zones, toggle works, and clicking a zone shows its popup.

Phase 5 — Inputs: search, map pick, time (4–5 h) `[FE]`

- [ ] **5.1 Geocoding helper** (`lib/geocode.js`) with the Mapbox Geocoding API (check the current endpoint in the Mapbox docs; v6 shown):

```
const T =
import.meta.env.VITE_MAPBOX_TOKEN;
export async function geocode(q) {
  const url =
`https://api.mapbox.com/search/geocode/
v6/forward?q=${encodeURIComponent(q)}`
    +
`&country=in&proximity=77.5946,12.9716&
limit=5&access_token=${T}`;
  const r = await fetch(url);
  if (!r.ok) throw new Error('Geocoding
failed');
  const d = await r.json();
  return d.features.map(f => ({
    label: f.properties.full_address ||
f.properties.name,
    coords: f.geometry.coordinates,
// [lng, lat]
```

```
  }));
}
```

- [ ] **5.2** `PlaceInput.jsx` : text field, 300 ms debounce, dropdown of suggestions, selecting sets `origin` or `destination` in the store.
- [ ] **5.3 Map-click picking:** buttons "Pick on map" for origin/destination; on map click, set the coordinates and label as "Dropped pin".
- [ ] **5.4 Markers:** green marker for origin, white/red for destination (`new mapboxgl.Marker()`).
- [ ] **5.5 Time-of-travel control:** `<input type="datetime-local">` defaulting to now, plus quick chips: **Now · 2 PM · 9 PM · 11 PM**. Save to `travelTime` .
- [ ] **5.6 Mode toggle** *(proposed)*: Drive / Walk (`mode` in store). Start with Drive; add Walk if time permits.
- [ ] **5.7 Demo button:** "Load demo trip" fills origin, destination and 10:30 PM from `shared/demo.js` . This is your safety net on stage.
- [ ] **5.8 Validation:** disable "Find safer route" until origin and destination are set and different; show inline errors for out-of-area points (outside a Bengaluru bounding box).
- [ ] **5.9 Submit button** "Find safer route" triggers the analysis pipeline (Phase 9).

Done when: you can search two places or click the map, set a time, and see both markers; the demo button fills everything instantly.

Phase 6 — Routing module (3–4 h)

`[FE/logic]`

- [ ] **6.1** `lib/routing.js` **— fetch routes** from the Mapbox Directions API:

```
const T =
import.meta.env.VITE_MAPBOX_TOKEN;

export async function
fetchRoutes(points, { profile =
'driving', alternatives = true } = {})
{
  const path = points.map(p =>
p.join(',')).join(';');
  const url =
`https://api.mapbox.com/directions/v5/m
apbox/${profile}/${path}`
    + `?
alternatives=${alternatives}&geometries
=geojson&overview=full&access_token=${T
}`;
  const r = await fetch(url);
  if (!r.ok) throw new
Error(`Directions failed:
${r.status}`);
```

```
  const d = await r.json();
  if (!d.routes?.length) throw new
Error('No route found');
  return d.routes.map((rt) => ({
    durationMin: rt.duration / 60,
    distanceKm: rt.distance / 1000,
    geometry: rt.geometry,
// GeoJSON LineString
  }));
}
```

Notes: alternatives are normally available only for two-point requests, so use `alternatives=false` when you pass extra waypoints (verify in the Mapbox docs). Walking alternatives may differ by profile.

- [ ] **6.2 Normalise** each route into `{ id, label, durationMin, distanceKm, geometry }` with ids like `r0`, `r1`.
- [ ] **6.3 Deduplicate** near-identical routes: compare duration (±0.5 min) and distance (±2%) or the Hausdorff-like check on sampled points. Remove duplicates before scoring.
- [ ] **6.4 Draw routes** (temporary, before scoring exists): one line layer per route, fastest in blue to confirm routing works. Fit the map to `turf.bbox(line)` with `map.fitBounds(bbox, { padding: 80 })`.

Done when: after choosing the demo pair, the map shows 1–3 real road routes with durations in the console.

Phase 7 — Spatial analysis with Turf.js (4–5 h) `[FE/logic]`

Approach: split each route into ~50 m chunks, take each chunk's midpoint, test which zone contains it, and sum length by risk class. Overlapping zones: highest risk wins.

- [ ] **7.1** `lib/spatial.js`:

```
import * as turf from '@turf/turf';

const RANK = { none: 0, green: 1,
yellow: 2, red: 3 };
const CHUNK_KM = 0.05;

export function analyseRoute(geometry,
zones) {
  const line =
turf.lineString(geometry.coordinates);
  const chunks = turf.lineChunk(line,
CHUNK_KM, { units: 'kilometers' });
  const exposureKm = { red: 0, yellow:
0, green: 0, none: 0 };
  const crossed = new Set();
```

```
  const segments =
chunks.features.map((seg) => {
    const km = turf.length(seg, {
units: 'kilometers' });
    const mid = turf.along(seg, km / 2,
{ units: 'kilometers' });
    let risk = 'none', zoneId = null;
    for (const z of zones.features) {
      if (RANK[z.properties.risk] >
RANK[risk] &&
turf.booleanPointInPolygon(mid, z)) {
        risk = z.properties.risk;
        zoneId = z.properties.id;
      }
    }
    exposureKm[risk] += km;
    if (zoneId) crossed.add(zoneId);
    seg.properties = { risk, km };
    return seg;
  });

  return {
    exposureKm,
    crossedZones: [...crossed],
    segments:
turf.featureCollection(segments),   //
used to colour the route on the map
  };
}
```

- [ ] **7.2 Performance guard:** pre-compute each zone's bbox once; skip `booleanPointInPolygon` when the midpoint is outside the bbox.

- [ ] **7.3 Unit tests** (`spatial.test.js`, Vitest): a straight line fully inside a square red zone → red exposure ≈ line length; a line fully outside → all `none`; a line half inside → about half.
- [ ] **7.4 Console check** with the demo route: red exposure should be > 0 for the fastest route.

Done when: `analyseRoute` returns the red-km exposure for the demo fastest route and tests pass.

Phase 8 — Risk engine (3–4 h)

`[logic]`

A **pure function module** (no network, no React) so you can tune and test it quickly.

- [ ] **8.1** `shared/scoringConfig.js` *(proposed values)*:

```
export const DEFAULT_CONFIG = {
  weights: { red: 3, yellow: 1.5,
green: 0.5, none: 0 },
  timeBands: [
    { id: 'day',     from: 6,  to: 17,
mult: 1.0 },
    { id: 'evening', from: 17, to: 21,
mult: 1.3 },
    { id: 'night',   from: 21, to: 6,
mult: 1.6 },
```

```
  ],
  penaltyMinPerUnit: 5,   // minutes of
"risk penalty" per weighted-km
  scoreDecay: 0.5,        // higher =
score drops faster with exposure
  minScoreGain: 10,       // show a
safer route only if it beats fastest by
this much
};
```

- [ ] **8.2** `lib/riskEngine.js`:

```
export function timeBand(date, cfg) {
  const h = date.getHours();
  return cfg.timeBands.find(b => b.from
< b.to ? (h >= b.from && h < b.to) : (h
>= b.from || h < b.to));
}

export function scoreRoute(route,
analysis, date, cfg) {
  const band = timeBand(date, cfg);
  const E =
Object.entries(analysis.exposureKm)
    .reduce((sum, [cls, km]) => sum +
km * (cfg.weights[cls] ?? 0) *
band.mult, 0);
  const riskPenaltyMin =
cfg.penaltyMinPerUnit * E;
  return {
    ...route,
    band: band.id,
    weightedExposure: E,
```

```
    riskPenaltyMin,
    cost: route.durationMin +
riskPenaltyMin,
    score: Math.max(0, Math.min(100,
Math.round(100 * Math.exp(-
cfg.scoreDecay * E)))),
  };
}

export function chooseRoutes(scored,
cfg) {
  const fastest = [...scored].sort((a,
b) => a.durationMin - b.durationMin)
[0];
  const best = [...scored].sort((a, b)
=> a.cost - b.cost)[0];
  const safe = best.id !== fastest.id
&& best.score - fastest.score >=
cfg.minScoreGain ? best : null;
  return { fastest, safe };
}
```

- [ ] **8.3 Worked example to sanity-check** (with the config above): fastest route has weighted exposure E ≈ 1.09 → score ≈ 58, penalty ≈ 5.4 min, cost ≈ 23.4; detour has E ≈ 0.12 → score ≈ 94, penalty ≈ 0.6 min, cost ≈ 21.6. The detour wins on cost and on score.
- [ ] **8.4 Calibrate honestly.** The deck shows 18 min / 58 and 21 min / 94. Get those numbers by **adjusting zone shapes and the demo pair**, and tuning `scoreDecay` / `penaltyMinPerUnit` in the

config, so they come out of the engine. Do **not** hard-code them in the UI. If a judge asks, you can show the formula.

- [ ] **8.5 Unit tests** (`riskEngine.test.js`): no exposure → score 100; more red km → lower score; night multiplier lowers the score vs. day for the same exposure; `chooseRoutes` returns `safe = null` when the gain is below the threshold.
- [ ] **8.6 Time sensitivity check:** same route, 2 PM vs 11 PM → different score. This is a headline feature, so verify it explicitly.

Done when: tests pass and the demo pair produces the pitch-like comparison at the demo time.

Phase 9 — Safe corridor generation (4–6 h) `[logic]`

Mapbox cannot exclude polygons, so you steer routes with **detour waypoints** *(proposed)*.

- [ ] **9.1 Find the problem zones:** from the fastest route's analysis, take zones in `crossedZones` whose risk is `red` (then `yellow` if no red).
- [ ] **9.2 Generate waypoint candidates around each zone:**

```
import * as turf from '@turf/turf';

export function
detourWaypoints(zoneFeature, bufferKm =
0.3) {
  const buffered =
turf.buffer(zoneFeature, bufferKm, {
units: 'kilometers' });
  const [minX, minY, maxX, maxY] =
turf.bbox(buffered);
  // four corners of the buffered box =
candidate points to route through
  return [[minX, minY], [minX, maxY],
[maxX, minY], [maxX, maxY]];
}
```

Better version (if time): take points along the buffered outline with `turf.along(turf.polygonToLine(buffered), d)` at 8 spots and keep those not inside any red zone.

- [ ] **9.3 Request detour routes:** for each waypoint `w` call `fetchRoutes([origin, w, destination], { alternatives: false })`. Limit to ~4–6 requests total; run them with `Promise.allSettled` and ignore failures.
- [ ] **9.4 Reject bad candidates:** discard any route that is > 1.6× the fastest duration or has a waypoint that sits inside a red zone.

- [ ] **9.5 Merge** with the original alternatives, deduplicate (Step 6.3), analyse every candidate (Phase 7), score every candidate (Phase 8), call `chooseRoutes`.
- [ ] **9.6 Label results:** `fastest.label = 'Fastest'`, `safe.label = 'Safe corridor'`; compute `deltaMin = safe.durationMin - fastest.durationMin`.
- [ ] **9.7 Fallback when no safe route exists:** if `safe` is null, show "Fastest route is already within safe limits" (or "No safer path found nearby") with the score — still useful, never a blank screen.
- [ ] **9.8 Orchestrator** `lib/pipeline.js` → `runAnalysis({ origin, destination, time, zones, config, mode })` that returns `{ fastest, safe, all }`. Keep UI code out of it.

Done when: one function call turns the demo pair into `{ fastest: 18 min/58, safe: 21 min/94 }` (or your calibrated numbers).

Phase 10 — Results UI (5–6 h) `[FE]`

- [ ] **10.1 Wire the submit button** → `setStatus('loading')` → `runAnalysis` → `setResults(...)` or `setStatus('error', message)`.

- [ ] **10.2 Loading state:** skeleton card plus a "Analysing route risk..." message; disable the button to avoid double submits.
- [ ] **10.3 Route layers on the map:**
  - Fastest route: drawn as risk-coloured segments using `analysis.segments` (red/yellow/green/none by `['get','risk']`) so **risky stretches are visible**.
  - Safe corridor: solid accent-green line, slightly thicker.
  - Inactive route: lower opacity. Active route: full opacity, drawn on top.
  - Add a casing (wider dark line under each) for contrast.
- [ ] **10.4 Fit bounds** to both routes after results.
- [ ] **10.5** `RouteComparisonCard.jsx`**:** two columns/cards (Fastest vs Safe corridor) each showing: duration, distance, Safety Score (big number with colour: <60 red, 60–79 yellow, ≥80 green), crossed zones count, and risk-km. Show the trade-off line: "**+3 min for +36 safety points**" (computed).
- [ ] **10.6 Risk callout** on the fastest route: "Crosses a high-risk zone: Demo Zone A" with a button that zooms to the crossing.
- [ ] **10.7** `SafeSwitchButton`**:** big accent button "Switch to safer route"; sets `activeRouteId` to

the safe route, updates the map styling and the card highlight instantly (no refetch). After switching, show "Safer route active · 21 min" with an "Undo" option.

- [ ] **10.8 Time-band indicator:** "Late night (×1.6 risk weighting)" so judges see why time matters. Changing the time and re-running updates the numbers.
- [ ] **10.9 Animate lightly:** route line draw-in or fade; keep it fast.
- [ ] **10.10 Empty and error states:** no route found, API error, out-of-area, with a retry button.

Done when: from "Load demo trip" → click → fastest route shows red segments, comparison card shows both scores, and Safe Switch changes the active route instantly.

Phase 11 — Backend API (3–4 h)

`[BE]`

- [ ] **11.1** `server/index.js`**:**

```
import 'dotenv/config';
import express from 'express';
import cors from 'cors';
import rateLimit from 'express-rate-
limit';
```

```
import { readFile } from
'node:fs/promises';
import { DEFAULT_CONFIG } from
'../shared/scoringConfig.js';

const app = express();
app.use(cors({ origin:
process.env.CORS_ORIGIN }));
app.use(express.json({ limit: '20kb'
}));

app.get('/api/health', (_, res) =>
res.json({ ok: true }));
app.get('/api/config', (_, res) =>
res.json(DEFAULT_CONFIG));
app.get('/api/zones', async (_, res) =>
{
  const txt = await readFile(new
URL('./data/zones.bengaluru.geojson',
import.meta.url), 'utf8');

res.type('application/json').send(txt);
});

app.listen(process.env.PORT || 3001);
```

- [ ] **11.2 Client loader** (`lib/api.js`) fetching `/api/config` and `/api/zones` on startup, falling back to `/zones.bengaluru.geojson` and `DEFAULT_CONFIG` if the server is down.
- [ ] **11.3 Error middleware** returning JSON `{ error }` ; never leak stack traces.

- [ ] **11.4 CORS** locked to the client origin (update after deploy).

Done when: the client loads zones and config from the server, and still works when the server is switched off (fallback).

Phase 12 — Gemini safety copilot (4–5 h) `[AI]` `[BE]` `[FE]`

- [ ] **12.1 Server endpoint** `POST /api/copilot`:

```
import { GoogleGenAI } from
'@google/genai';
const ai = new GoogleGenAI({ apiKey:
process.env.GEMINI_API_KEY });

const SYSTEM = `You are SafeRoute
Guardian's safety copilot for a demo
app.
Use ONLY the JSON context provided.
Never invent incidents, statistics or
places.
The risk data is synthetic demo data;
say so briefly if asked about accuracy.
Say "safer", never "safe" or
"guaranteed". Be concise: max 4 short
sentences.
Explain trade-offs in minutes and
Safety Score. If asked something
```

```
outside route safety
or without data in the context, say you
can't tell from the available data.
Never give instructions that replace
emergency services; in real danger
advise contacting local emergency
services.`;

const copilotLimiter = rateLimit({
windowMs: 60_000, max: 20 });

app.post('/api/copilot',
copilotLimiter, async (req, res) => {
  try {
    const { question, context } =
req.body ?? {};
    if (typeof question !== 'string' ||
!question.trim() || question.length >
500)
      return res.status(400).json({
error: 'Invalid question' });

    const r = await
ai.models.generateContent({
      model: process.env.GEMINI_MODEL,
      contents:
`CONTEXT_JSON:\n${JSON.stringify(contex
t).slice(0,
6000)}\n\nQUESTION:\n${question}`,
      config: { systemInstruction:
SYSTEM, temperature: 0.3,
maxOutputTokens: 300 },
    });
    res.json({ answer: r.text });
```

```
  } catch (e) {
    res.status(502).json({ error:
'Copilot unavailable' });
  }
});
```

Check the current `@google/genai` usage and model names in Google's docs if a call fails; SDK details change.

- [ ] **12.2 Build the context object** on the client (small and structured):

```
{
  trip: { origin: 'A', destination:
'B', timeBand: 'night' },
  fastest: { durationMin: 18, score:
58, crossedZones: [{ name: 'Demo Zone
A', risk: 'red', note: '...' }] },
  safe:    { durationMin: 21, score:
94, crossedZones: [] },
  deltaMin: 3
}
```

- [ ] **12.3 `CopilotPanel.jsx`:** chat-style panel; suggested chips: "Why is the fastest route risky?", "Is +3 min worth it?", "What should I do if I feel unsafe?". Show a typing indicator; append answers to `copilot` in the store.
- [ ] **12.4 Auto-explanation:** after results arrive, request one short automatic summary (optional,

cache it so it does not refire on every render).

- [ ] **12.5 Template fallback** when the endpoint fails: generate a deterministic sentence on the client, e.g. "The fastest route crosses {zone} ({risk}). The safer corridor adds {n} min and raises the score from {a} to {b}." The demo never breaks.
- [ ] **12.6 Guardrail tests (manual):** ask something off-topic ("who won the match?") → it declines; ask "is it 100% safe?" → it says no guarantee; ask about an area with no data → "can't tell".

Done when: three sample questions get correct, grounded answers, and unplugging the internet still shows the template explanation.

Phase 13 — SOS workflow simulator (2–3 h) `[FE]`

- [ ] **13.1** `SOSSimulator.jsx` — floating SOS button (bottom-right) opening a modal labelled "**SIMULATION — no real alerts are sent**".
- [ ] **13.2 State machine:** `idle → countdown (5 s, cancellable) → alert_sent → contacts_notified → live_location_shared → help_dispatched (simulated)`. Use `useReducer` and timeouts; clean up timers on unmount.

- [ ] **13.3 Show the simulated outputs:** fake contact cards ("Contact 1 — notified"), the current route position on the map, a timeline with timestamps.
- [ ] **13.4 Cancel / reset** buttons; keyboard accessible.
- [ ] **13.5 Guarantee no network calls:** check the browser Network tab during a simulation (must be zero requests to anything external).

Done when: a 20-second simulated flow plays end to end, can be cancelled, and is visibly labelled as a simulation.

Phase 14 — Resilience and demo mode (3–4 h) `[FE]` `[BE]`

- [ ] **14.1 Cache results.** After a successful demo run, store `{ fastest, safe, all }` in `localStorage` keyed by `origin|destination|bandId`. Reuse on failure.
- [ ] **14.2 Bundle a recorded demo dataset:** save the demo's Directions JSON responses into `client/public/demo-routes.json`.
- [ ] **14.3 `?demo=1` mode:** skip Mapbox Directions calls, load the cached routes, still run the real spatial and risk engine locally. Great for bad Wi-Fi on stage.

- [ ] **14.4 Timeouts:** wrap fetches with `AbortController` (8–10 s) and show a friendly error.
- [ ] **14.5 Mapbox quota awareness:** avoid re-fetching on every state change; only fetch on submit.
- [ ] **14.6 Error boundary** at the app root so a crash shows a recoverable screen, not a blank page.

Done when: with Wi-Fi off and `?demo=1`, the full demo still works (map tiles need the network, so keep a screen recording as the final backup).

Phase 15 — Polish and accessibility (4–5 h) `[FE]`

- [ ] **15.1 Responsive pass** at 360 px, 768 px, 1280 px, 1920 px. Mobile: bottom sheet, large tap targets (≥44 px).
- [ ] **15.2 Visual consistency:** one font pair (e.g., a pixel-style display font for headings and a monospace/sans for body to echo the deck), spacing scale, consistent radius.
- [ ] **15.3 Accessibility:** colour is never the only signal (add icons/labels like "High risk"), focus rings, `aria-label`s on map controls and buttons,

contrast ≥ 4.5:1, keyboard operable inputs and SOS modal.

- [ ] **15.4 Microcopy:** use "safer" not "safe"; show "synthetic demo data" in the legend and footer; short helpful hints under inputs.
- [ ] **15.5 Favicon, title, meta tags**, and a clean landing/hero state with the tagline.
- [ ] **15.6 Performance:** Lighthouse check; lazy-load the copilot and SOS components; confirm the first screen is interactive in < 3 s on a normal connection.
- [ ] **15.7 Remove console noise and dead code.**

Done when: a teammate who has never seen the app can complete the demo flow unaided on their phone.

Phase 16 — Testing and QA (3–4 h)

`[QA]`

- [ ] **16.1 Automated:** `npm run test` in `client` for spatial and risk-engine tests; run in CI (GitHub Actions) if time permits.
- [ ] **16.2 Manual test matrix:**

#	Scenario	Expected
1	Demo trip, 10:30 PM	Fastest has red exposure; safe corridor higher score, small time delta
2	Same trip, 2 PM	Scores differ from night (time effect visible)
3	Trip entirely in green/none areas	No safer route needed; clear message
4	Origin = destination	Validation error
5	Point outside Bengaluru	Out-of-area message
6	Directions API fails	Cached/error message, no crash
7	Gemini fails	Template explanation shown
8	Safe Switch then Undo	Map and card update both ways
9	Zone toggle on/off	Overlay shows/hides

#	Scenario	Expected
10	SOS simulation	Completes, cancellable, zero network calls
11	Mobile 360 px	No horizontal scroll, controls reachable
12	Keyboard only	Whole flow operable

- [ ] **16.3 Cross-browser:** Chrome, Safari (iOS), Firefox.
- [ ] **16.4 Security check:** no keys in the client bundle (search the build output for `GEMINI`), CORS blocks other origins, rate limit works.
- [ ] **16.5 Bug bash:** each member tries to break it for 20 minutes; log and fix.

Done when: all 12 rows pass on the deployed URL, not only on localhost.

Phase 17 — Deployment (2–3 h)

`[OPS]`

- [ ] **17.1 Deploy the server** (Render/Railway): root `server`, start command `node index.js`; set `GEMINI_API_KEY`, `GEMINI_MODEL`, `PORT`,

`CORS_ORIGIN`. Note: free tiers sleep, so ping `/api/health` a few minutes before the demo.

- [ ] **17.2 Deploy the client** (Vercel/Netlify): root `client`, build `npm run build`, output `dist`; set `VITE_MAPBOX_TOKEN` and `VITE_API_BASE=<server URL>`.
- [ ] **17.3 Update CORS** on the server to the client's final URL.
- [ ] **17.4 Restrict the Mapbox token** to the deployed domain(s) plus `localhost` in the Mapbox account.
- [ ] **17.5 Smoke-test the live site** with the full matrix on at least two devices.
- [ ] **17.6 Tag the release** (`v1.0-demo`) and freeze `main` except for fixes.

Done when: a public URL works end to end from a phone on mobile data.

Phase 18 — Demo and pitch preparation (3–4 h) `[OPS]`

- [ ] **18.1 Write the 3-minute demo script:**
  1. *(20 s)* Problem: "Navigation asks how fast, not how safe."

2. *(30 s)* Load demo trip at night → fastest route 18 min, red segments, score 58.
3. *(30 s)* Show the safe corridor: +3 min, score 94. Explain **Route Cost = Travel Time + Risk Penalty**.
4. *(20 s)* Change time to 2 PM → score changes (time-aware).
5. *(25 s)* One-tap Safe Switch.
6. *(30 s)* Ask the copilot "Is +3 minutes worth it?".
7. *(15 s)* SOS simulation (labelled as a simulation).
8. *(10 s)* Close with the tagline and what's next.

- [ ] **18.2 Record a backup video** (1080p, with audio) of the full flow; keep it on the laptop and phone.
- [ ] **18.3 Prepare Q&A answers:**
  - *Where does the data come from?* Synthetic for the demo; production would use public datasets, lighting/footfall data, and community reports.
  - *Why not just use Google Maps?* It optimises time; we make safety part of the route cost and offer an actionable alternative.
  - *How accurate are the scores?* Demo scores come from a transparent formula over

synthetic zones; real use needs validated data.
    - *Privacy?* No location storage in the MVP.
    - *Scalability?* Zones become a spatial database; scoring is a pure function; swap Mapbox for any routing engine.
    - *Is SOS real?* No, simulated by design.

- [ ] **18.4 Rehearse 3 times** with a timer; assign who clicks and who speaks; rehearse the failure plan (switch to `?demo=1`, then the video).
- [ ] **18.5 Pre-flight checklist (30 min before):** laptop charged, hotspot ready, server pinged awake, tab pre-loaded at the demo URL, browser zoomed for projector, notifications off.

Done when: the team can deliver the demo under 3 minutes, twice in a row, without looking at notes.

Suggested Timeline (adjust to your hackathon length)

Block	Phases
Foundation	0, 1, 2, 3
Core engine (critical path)	4, 5, 6, 7, 8, 9

Block	Phases
Product surface	10, 11, 12, 13
Hardening	14, 15, 16
Ship and present	17, 18

If time is tight, **cut in this order:** Walk mode → auto copilot summary → waypoint outline refinement → animations → Lighthouse tuning. **Never cut:** demo mode, time-of-day effect, Safe Switch, the synthetic-data label.

Definition of Done (whole project)

- [ ] Demo trip shows fastest vs safe corridor with scores and time delta, produced by the real engine.
- [ ] Changing the time of travel changes the scores.
- [ ] Risky segments are visibly highlighted on the fastest route.
- [ ] One-tap Safe Switch works instantly and can be undone.
- [ ] Copilot answers are grounded; fallback explanation works.

- [ ] SOS simulator is clearly labelled and makes no external calls.
- [ ] Synthetic-data disclaimer is visible in the UI.
- [ ] No API keys exposed in the client; Gemini key lives only on the server.
- [ ] Deployed URL works on mobile; `?demo=1` fallback and backup video ready.

Common Pitfalls

- **Coordinates reversed.** GeoJSON and Mapbox use `[lng, lat]`; many tools show `lat, lng`. Wrong order puts your zones in the ocean.
- **Unclosed polygons.** The first and last coordinate must match.
- **Alternatives with waypoints.** Directions alternatives generally work only for two-point requests; use separate calls for detours.
- **Hard-coded scores.** Judges may probe; keep numbers engine-derived.
- **Layer errors on reload.** Add sources and layers only after `map.on('load')` and check `map.getLayer(id)` before adding or removing.
- **React StrictMode double mount** can create two maps in development; always clean up with `map.remove()`.

- **Free server cold starts** causing a slow first request during the demo; ping early.
- **Overclaiming safety.** Use "safer", keep the synthetic-data notice, and keep the SOS "simulation" label.